Lab 5: Static Code Analysis

Oct 28, 2026 · @Dhruva

Overview

After fixes, `blood_donation.py` scores **10.00/10 on Pylint** (up from 4.81), with **0 Bandit issues** (down from 4) and **0 Flake8 errors** (down from 4). Its demo output is unchanged.

Field	Details
Name	Dhruva Myakeri
SRN	PES1UG24CS156
Lab	Lab 5: Static Code Analysis
Assigned repo	SETAPESU26/L5_28_blood_donation
File analysed	`blood_donation.py`

Objective: improve the quality, security and style of a Python script using Pylint, Bandit and Flake8 to find and fix common problems.

Environment setup

Tool	Version	Role
Python	3.14.3	Interpreter (virtual env `venv`)
Pylint	4.1.3	Code quality and logical errors
Bandit	1.9.4	Security vulnerabilities
Flake8	7.4.1	PEP 8 style and formatting

```
python -m venv venv
venv\Scripts\activate          # Windows (Linux/macOS: source
venv/bin/activate)
pip install pylint bandit flake8
```

Step 1: Inspect the program

`blood_donation.py` is a donor register for a blood bank. It:

- registers donors after an age check (18–65)
- decides who can donate again (a 90-day gap)
- records donations into stock for each blood group
- counts the compatible units available for a patient

The helpers `donorSheetPath` and `notifyDonors` are not called by the demo.

Command: `python blood_donation.py`. The output below is identical before and after the fixes:

```
Stock: {'A+': 0, 'A-': 0, 'B+': 0, 'B-': 0, 'O+': 2, 'O-': 1, 'AB+': 0, 'AB-
': 0}
Units for an A+ patient: 3
Units for a Z+ patient: 0
Is O- universal? True
Low stock groups: ['A+', 'A-', 'B+', 'B-', 'O-', 'AB+', 'AB-']
Donors eligible today: []
```

Priya donated 30 days ago, so the 90-day rule blocks her donation. Old Donor (age 70) is rejected at registration. Arjun and Farah donate today, so nobody is eligible at the end of the run.

Step 2: Run the static analysis tools (before)

The original file scored **4.81/10** on Pylint, with 4 Bandit issues (1 High, 1 Medium, 2 Low) and 4 Flake8 errors. The reports are saved in `reports/before/`.

```
pylint blood_donation.py > pylint_report.txt
bandit -r blood_donation.py > bandit_report.txt
flake8 blood_donation.py > flake8_report.txt
```

pylint_report.txt (before)

```
************* Module blood_donation
blood_donation.py:1:0: C0114: Missing module docstring (missing-module-
docstring)
blood_donation.py:17:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:17:0: C0103: Function name "registerDonor" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:17:18: C0103: Argument name "donorName" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:17:29: C0103: Argument name "bloodGroup" doesn't conform to
snake_case naming style (invalid-name)
```

```
blood_donation.py:17:46: C0103: Argument name "lastDonation" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:17:0: W0102: Dangerous default value [] as argument
(dangerous-default-value)
blood_donation.py:27:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:27:0: C0103: Function name "canDonate" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:33:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:33:0: C0103: Function name "recordDonation" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:33:19: C0103: Argument name "donorName" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:42:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:42:0: C0103: Function name "unitsFor" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:42:13: C0103: Argument name "patientGroup" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:45:11: W0718: Catching too general exception Exception
(broad-exception-caught)
blood_donation.py:49:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:49:0: C0103: Function name "isUniversal" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:49:16: C0103: Argument name "bloodGroup" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:50:11: R1714: Consider merging these comparisons with 'in'
by using 'bloodGroup in ('O-', 'AB+')'. Use a set instead if elements are
hashable. (consider-using-in)
blood_donation.py:53:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:53:0: C0103: Function name "lowStock" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:58:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:58:0: C0103: Function name "donorSheetPath" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:63:0: C0116: Missing function or method docstring (missing-
function-docstring)
blood_donation.py:63:0: C0103: Function name "notifyDonors" doesn't conform
to snake_case naming style (invalid-name)
blood_donation.py:63:17: C0103: Argument name "phoneList" doesn't conform to
snake_case naming style (invalid-name)
blood_donation.py:68:0: C0116: Missing function or method docstring (missing-
```

```
function-docstring)

-----------------------------------
Your code has been rated at 4.81/10
```

bandit_report.txt (before)

```
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password:
'bloodbank-demo'
   Severity: Low   Confidence: Medium   CWE: CWE-259
   Location: .\blood_donation.py:5:19
5       OFFICER_PASSWORD = "bloodbank-demo"
--------------------------------------------------
>> Issue: [B101:assert_used] Use of assert detected. The enclosed code will
be removed when compiling to optimised byte code.
   Severity: Low   Confidence: High   CWE: CWE-703
   Location: .\blood_donation.py:18:4
18          assert bloodGroup in stock, "Unknown blood group"
--------------------------------------------------
>> Issue: [B306:blacklist] Use of insecure and deprecated function (mktemp).
   Severity: Medium   Confidence: High   CWE: CWE-377
   Location: .\blood_donation.py:60:11
60          return tempfile.mktemp(suffix=".csv")
--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell,
possible injection detected, security issue.
   Severity: High   Confidence: High   CWE: CWE-78
   Location: .\blood_donation.py:65:4
65          os.system("send-sms " + " ".join(phoneList))
--------------------------------------------------
Total issues (by severity): Undefined: 0  Low: 2  Medium: 1  High: 1
```

flake8_report.txt (before)

```
blood_donation.py:7:80: E501 line too long (82 > 79 characters)
blood_donation.py:58:1: E303 too many blank lines (3)
blood_donation.py:80:43: E261 at least two spaces before inline comment
blood_donation.py:81:13: E225 missing whitespace around operator
```

Step 3: Identified issues

I fixed 15 issues, high and medium severity first. Two of them (#5 and #15) were not reported by any tool; I found them while reading the code. Line numbers refer to the original file.

#	Issue	Type	Line(s)	Description	Fix approach
1	Shell injection — Bandit B605 (High)	Security	65	`os.system("send-sms " + " ".join(phoneList))` passes caller data to a shell, so `"123; rm -rf /"` would run an extra command.	Validate each number with a regex, resolve the tool with `shutil.which`, and call `subprocess.run([tool, *phone_list], check=True)` with no shell.
2	Insecure temp file — Bandit B306 (Medium)	Security	60	`tempfile.mktemp()` only returns a name, so another process can create that file first (race condition, CWE-377).	Use `tempfile.mkstemp()`, which creates the file atomically and privately; close the descriptor and return the path.
3	Mutable default argument — Pylint W0102	Bug	17	`notes=[]` is created once and shared by every call, so notes pile up across calls.	Default to `notes=None` and create a new list inside the function.
4	Broad exception — Pylint W0718	Bug	45–46	`except Exception` swallows every error, including the bug in #5.	Catch only `KeyError` (unknown blood group).
5	Incomplete compatibility table (code review)	Bug	8–14	`COMPATIBLE` had no `A-`, `B-` or `AB-` keys, so those patients always got 0 units.	Add `A-: [A-, O-]`, `B-: [B-, O-]`, `AB-: [AB-, A-, B-, O-]`.
6	Hardcoded password — Bandit B105 (Low)	Security	5	`OFFICER_PASSWORD = "bloodbank-demo"` puts a secret in source control.	Read it with `os.environ.get("OFFICER_PASSWORD", "")`.
7	Assert used for validation — Bandit B101 (Low)	Security	18	`assert` is removed under `python -O`, so the blood-group check disappears.	`if blood_group not in stock: raise ValueError(...)`
8	Naming — Pylint C0103 (×19)	Convention	17–63	camelCase names (`registerDonor`, `donorName`, …)	Rename to snake_case and update the call sites.
9	Missing docstrings — Pylint C0114/C0116 (×9)	Convention	1, 17–68	No module or function docstrings.	Add a module docstring and one per function.
10	Chained comparison — Pylint R1714	Refactor	50	`bloodGroup == "O-" or bloodGroup == "AB+"`	`blood_group in ("O-", "AB+")`
11	Line too long — Flake8 E501	Style	7	`stock` dict line is 82 characters (limit 79).	Split the dict across lines.

#	Issue	Type	Line(s)	Description	Fix approach
12	Too many blank lines — Flake8 E303	Style	56–58	Three blank lines before a function.	Reduce to two.
13	Inline comment spacing — Flake8 E261	Style	80	One space before `# below 2 units`.	Use two spaces.
14	Operator whitespace — Flake8 E225	Style	81	`eligible=[...]`	`eligible = [...]`
15	Code runs on import (code review)	Maintainability	85	`main()` ran at module level, so importing the module ran the demo.	Wrap it in `if __name__ == "__main__":`.

Step 4: Fix the issues

I made one commit per tool, after a baseline commit of the original reports. After each commit I reran the demo and checked that its output had not changed.

Commit	Message	Issues fixed
`94af526`	Add baseline Pylint, Bandit and Flake8 reports	— (`reports/before/`)
`f9e5305`	Fix security issues reported by Bandit	#1, #2, #6, #7
`62ccd29`	Fix code quality issues reported by Pylint	#3, #4, #5, #8, #9, #10, #15
`ac84951`	Fix PEP 8 style issues reported by Flake8	#11–#14
`bd959df`	Add after-fix reports, issue table and reflection	— (`reports/after/`, `issues.md`, `reflection.md`)

Commit 1 — Bandit (security)

Before:

```
OFFICER_PASSWORD = "bloodbank-demo"
...
    assert bloodGroup in stock, "Unknown blood group"
...
    return tempfile.mktemp(suffix=".csv")
...
    os.system("send-sms " + " ".join(phoneList))
```

After:

```
OFFICER_PASSWORD = os.environ.get("OFFICER_PASSWORD", "")
PHONE_PATTERN = re.compile(r"\+?\d{7,15}")
...
    if blood_group not in stock:
        raise ValueError("Unknown blood group: " + str(blood_group))
...
    fd, path = tempfile.mkstemp(suffix=".csv")
    os.close(fd)
    return path
...
    for phone in phone_list:
        if not PHONE_PATTERN.fullmatch(phone):
            raise ValueError("Invalid phone number: " + repr(phone))
    sms_tool = shutil.which("send-sms")
    if sms_tool is None:
        raise FileNotFoundError("send-sms command not found")
    # Arguments are passed as a list (no shell) and validated above.
    subprocess.run([sms_tool, *phone_list], check=True)  # nosec B603
```

I tested the new helpers directly. An injected value such as `'9876543210; rm -rf /'` is now rejected with `ValueError`, and `donor_sheet_path()` creates a real temp file.

After the fix, Bandit raised two generic Low reminders: B404 (importing `subprocess`) and B603 (subprocess call without a shell). I reviewed both and suppressed them with targeted `# nosec` comments. No shell is used, the arguments are a list and every number is validated, so no injection path remains.

Commit 2 — Pylint (quality)

Before:

```
def registerDonor(donorName, bloodGroup, age, lastDonation=None, notes=[]):
...
    except Exception:
        return 0
...
    return bloodGroup == "O-" or bloodGroup == "AB+"
...
main()
```

After:

```
def register_donor(donor_name, blood_group, age, last_donation=None,
                   notes=None):
    """Register a donor aged 18-65; return True if they were added."""
    if notes is None:
        notes = []
...
    except KeyError:
        return 0
...
    return blood_group in ("O-", "AB+")
...
if __name__ == "__main__":
    main()
```

I also added the missing `A-`, `B-` and `AB-` entries to `COMPATIBLE`. With test stock `A-=1, B-=2, AB-=3, O-=4`, the counts are now A- = 5, B- = 6 and AB- = 10. Before the fix, all three returned 0.

Commit 3 — Flake8 (style)

```
# Before
stock = {"A+": 0, "A-": 0, "B+": 0, "B-": 0, "O+": 0, "O-": 0, "AB+": 0, "AB-
": 0}
print("Low stock groups:", lowStock()) # below 2 units
eligible=[donor["name"] for donor in donors if canDonate(donor)]

# After
stock = {
    "A+": 0, "A-": 0, "B+": 0, "B-": 0,
    "O+": 0, "O-": 0, "AB+": 0, "AB-": 0,
}
print("Low stock groups:", low_stock())  # below 2 units
eligible = [donor["name"] for donor in donors if can_donate(donor)]
```

I also removed the third blank line before `donor_sheet_path` (E303).

Verification (after)

I reran all three tools on the final file. Every issue I targeted is gone, and Pylint is above the 9.5 target at 10.00/10. The reports are saved in `reports/after/`.

Tool	Before	After
Pylint	4.81/10 (28 messages)	**10.00/10** (0 messages)
Bandit	4 issues (1 High, 1 Medium, 2 Low)	**0 issues** (2 reviewed `# nosec`)
Flake8	4 errors	**0 errors** (empty report)
Demo output	6 lines (Step 1)	Identical

pylint_report.txt (after)

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

bandit_report.txt (after)

```
Test results:
        No issues identified.

Code scanned:
        Total lines of code: 94
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled
(e.g., #nosec BXXX): 2

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
```

flake8_report.txt (after)

The file is empty. Flake8 prints nothing when there are no violations.

Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

Easiest: the Flake8 style errors (E501, E303, E261, E225) and Pylint's naming and docstring warnings. Each one points to an exact line and has one obvious fix that cannot change how the program behaves.

Hardest: the Bandit B605 shell injection. Swapping `os.system` for `subprocess` was not enough, because Bandit then raised B404 and B603. I had to make the call truly safe: no shell, a list of arguments, a resolved executable path and regex-validated phone numbers.

The broad `except Exception` (W0718) was also harder than it looked. Narrowing it to `KeyError` led me to a hidden bug: `COMPATIBLE` had no A-, B- or AB- entries, so those patients always got 0 units.

2. Did the static analysis tools report any false positives? If so, describe one example.

Yes. Bandit B603 ("subprocess call without shell") appeared after my fix. It is raised for every subprocess call, even one with no shell, list arguments and validated input. B404 (importing `subprocess`) is the same kind of blanket warning. I reviewed both and suppressed them with targeted `# nosec` comments.

B105 was borderline: it flagged `OFFICER_PASSWORD` only because of its name. The value was an unused demo string, but I moved it to an environment variable anyway.

The tools also missed real problems: the incomplete compatibility table, and `main()` running on import. Static analysis does not replace reading the code.

3. How would you integrate static analysis tools into your actual software development workflow?

- **While coding:** turn on Pylint and Flake8 in the editor so warnings appear as I type.
- **Before each commit:** a `pre-commit` hook that runs `flake8`, `pylint` and `bandit -r .`.
- **In CI:** run all three tools in GitHub Actions on every push and pull request. Fail the build on any Bandit Medium/High issue, any Flake8 error, or `pylint --fail-under=9.5`.
- **Shared config:** keep the rules in `pyproject.toml` / `.flake8`, and require a written reason for every `# nosec` or `# pylint: disable`.

4. What tangible improvements did you observe in code quality, readability, or robustness after applying the fixes?

- **Scores:** Pylint went from 4.81 to 10.00/10, Bandit from 4 issues to 0, and Flake8 from 4 errors to 0. The demo output did not change.
- **Security:** user input can no longer run shell commands, temp files are created atomically, no secret is stored in source, and validation still works under `python -O`.
- **Correctness:** there is no shared mutable default, and A-, B- and AB- patients now get correct unit counts.
- **Robustness:** unexpected errors are no longer swallowed by a broad `except`.
- **Readability:** snake_case names, docstrings, PEP 8 formatting, and a `__main__` guard so the module can be imported and tested.

Appendix: final blood_donation.py

```
"""Blood donation management: donor register, eligibility and stock."""

import os
import re
import shutil
# subprocess is only used without a shell, see notify_donors.
import subprocess  # nosec B404
import tempfile
from datetime import date, timedelta

OFFICER_PASSWORD = os.environ.get("OFFICER_PASSWORD", "")
PHONE_PATTERN = re.compile(r"\+?\d{7,15}")
donors = []
stock = {
    "A+": 0, "A-": 0, "B+": 0, "B-": 0,
    "O+": 0, "O-": 0, "AB+": 0, "AB-": 0,
}
COMPATIBLE = {
    "A+": ["A+", "A-", "O+", "O-"],
    "A-": ["A-", "O-"],
    "B+": ["B+", "B-", "O+", "O-"],
    "B-": ["B-", "O-"],
    "O+": ["O+", "O-"],
    "O-": ["O-"],
    "AB+": list(stock),
    "AB-": ["AB-", "A-", "B-", "O-"],
}
```

```
def register_donor(donor_name, blood_group, age, last_donation=None,
                   notes=None):
    """Register a donor aged 18-65; return True if they were added."""
    if notes is None:
        notes = []
    if blood_group not in stock:
        raise ValueError("Unknown blood group: " + str(blood_group))
    if age < 18 or age > 65:
        notes.append(donor_name + " is not eligible because of age")
        return False
    donors.append({"name": donor_name, "group": blood_group,
                   "last": last_donation})
    return True


def can_donate(donor):
    """Return True if at least 90 days have passed since the last
donation."""
    if donor["last"] is None:
        return True
    return date.today() - donor["last"] >= timedelta(days=90)


def record_donation(donor_name, units=1):
    """Add units to stock for an eligible donor; return True on success."""
    for donor in donors:
        if donor["name"] == donor_name and can_donate(donor):
            stock[donor["group"]] += units
            donor["last"] = date.today()
            return True
    return False


def units_for(patient_group):
    """Return the number of compatible units in stock for a patient."""
    try:
        return sum(stock[group] for group in COMPATIBLE[patient_group])
    except KeyError:
        return 0


def is_universal(blood_group):
    """Return True for O- (universal donor) or AB+ (universal recipient)."""
    return blood_group in ("O-", "AB+")
```

```
def low_stock(threshold=2):
    """Return the blood groups whose stock is below the threshold."""
    return [group for group, units in stock.items() if units < threshold]


def donor_sheet_path():
    """Create a private temporary CSV file and return its path."""
    # Helper kept for the lab: not called by the demo
    fd, path = tempfile.mkstemp(suffix=".csv")
    os.close(fd)
    return path


def notify_donors(phone_list):
    """Send an SMS to each phone number using the send-sms tool."""
    # Helper kept for the lab: not called by the demo
    for phone in phone_list:
        if not PHONE_PATTERN.fullmatch(phone):
            raise ValueError("Invalid phone number: " + repr(phone))
    sms_tool = shutil.which("send-sms")
    if sms_tool is None:
        raise FileNotFoundError("send-sms command not found")
    # Arguments are passed as a list (no shell) and validated above.
    subprocess.run([sms_tool, *phone_list], check=True)  # nosec B603


def main():
    """Run a short demonstration of the donor register."""
    register_donor("Arjun", "O+", 25)
    register_donor("Priya", "A+", 30, date.today() - timedelta(days=30))
    register_donor("Old Donor", "B+", 70)
    register_donor("Farah", "O-", 41)
    record_donation("Arjun", 2)
    record_donation("Priya")
    record_donation("Farah")
    print("Stock:", stock)
    print("Units for an A+ patient:", units_for("A+"))
    print("Units for a Z+ patient:", units_for("Z+"))
    print("Is O- universal?", is_universal("O-"))
    print("Low stock groups:", low_stock())  # below 2 units
    eligible = [donor["name"] for donor in donors if can_donate(donor)]
    print("Donors eligible today:", eligible)
```

```
if __name__ == "__main__":
    main()
```